Week 8 Tutorial

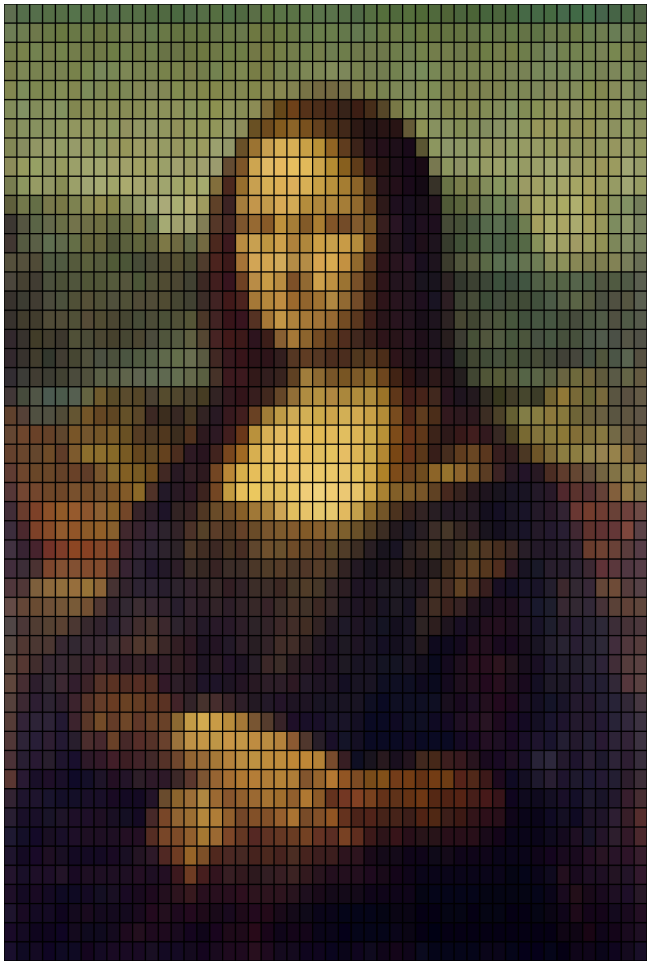
******For next week's tutorial session, please ensure you form groups of 3 or 4 members. All members of a group should be from the same tutorial session.******

******Enter your group members into a group on the assignment groups section of the people tab of this units Canvas page.******

Today's tutorial is going to cover a basic way of representing an image using code, and will teach us about using classes and OOP (object-oriented programming) in p5.js. This tutorial material is a valid basis for your assignment, however if you choose to use this code with only minor modifications you will not be able to get an HD in some parts of the rubric, however you can still do well with this as a start.

Today we'll convert an image into manipulable objects, representing it using coloured rectangles. We'll use Leonardo Da Vinci's 'Mona Lisa' for this exercise.

We will take the original image and use coloured rectangles to represent it like this:



Preparation:

You will need to get a copy of the image we are using, you can find it **here (https://canvas.sydney.edu.au/courses/72128/files/47943964/preview)** . To make things easy don't change the name of the file when you download it.

- Create a new p5.js sketch in VSCode using the p5.vscode plugin.
- Create a new folder inside the sketch folder called assets.
- Move the downloaded image into the assets folder.
- Now we are ready for the tutorial.

Part 1:

We're using a resized version of the image, which is 500x700 pixels. This allows us to easily represent it with rectangles. We can either use 10x10 pixel squares (resulting in 50x70 segments) or 10x14 pixel rectangles (resulting in 50x50 segments). We'll opt for the latter for simplicity as that means that the rectangles we use are the same shape as the overall image, and there is the same number of rectangles vertically as there is horizontally.

Objective:

For the first part of the tutorial we are going to make the segments we need later, using a segment class that stores x and y positions and has draw a function. There will be 2500 (50*50) segments, that will the same shape as the total image and will each have a unique position so that when they are all drawn they cover the image correctly. The instances of the segment class will be stored in an array in rows from left to right and from top to bottom.

Challenges:

- Creating a class that efficiently represents each segment with properties for position and size and a draw function.
- Calculating the correct width and height of the segments.
- Calculating the x and y positions of each segment.
- Creating the segments and storing them in an array.
- Drawing the segments back over the image.

Desired Results:

- A cohesive program that breaks down an image into 2500 individual segments and then draws each segment at its appropriate position on the canvas, effectively reassembling the original image.
- The segments should seamlessly align to display the complete image, with each segment acting as a puzzle piece in the larger picture.
- A visual demonstration of object-oriented programming principles through the use of a class to encapsulate segment properties and behaviors.

Solutions:

- Implement a "Segment" class with properties for x and y positions, width, and height, and a method for drawing the segment. These properties are set with the class constructor.
- Calculate the width and height of the segments based on the total dimensions of the canvas and the desired number of segments, ensuring equal distribution across the canvas, we can use the image properties from p5.js to get the dimensions.
- We will use nested for loops to create the segment in our case the outer loop goes over the height of the image. The outer loop will run once (not reaching its limit), addressing the first row. Then the inner loop will run until its limit, traversing a whole row. Then the outer loop will advance once more, triggering the inner loop to run until its limit, traversing the second row. This will continue until the outer loop reaches its limit.
- We will use a flat array to store segment instances, allowing for easy access and systematic drawing of segments. The order of elements in our array is not vitally important as each of the objects has its correct position stored internally.

Implementation:

```
//We need a variable to hold our image
let img;

//We will divide the image into segments, this is the number of segments in each dimension
//The total number of segments will be 2500 (50 * 50)

let numSegments = 50;

//We will store the segments in an array
let segments = [];

//lets load the image from disk
function preload() {
  img = loadImage('assets/Mona_Lisa_by_Leonardo_da_Vinci_500_x_700.jpg');
}

function setup() {
  //We will make the canvas the same size as the image using its properties
  createCanvas(img.width, img.height);
  //We can use the width and height of the image to calculate the size of each segment
  let segmentWidth = img.width / numSegments;
  let segmentHeight = img.height / numSegments;
  /*
  Divide the original image into segments, we are going to use nested loops
  let's have a look at how nested loops work
  first we use a loop to move down the image,
  we will use the height of the image as the limit of the loop
  then we use a loop to move across the image,
  we will use the width of the image as the limit of the loop
  Let's look carefully at what happens, the outer loop runs once, so we start at the top of th
e image
  Then the inner loop runs to completion, moving from left to right across the image
  Then the outer loop runs again, moving down 1 row image, and the inner loop runs again,
  moving all the way from left to right
  */

  for (let segYPos=0; segYPos<img.height; segYPos+=segmentHeight) {
    //this is looping over the height
    for (let segXPos=0; segXPos<img.width; segXPos+=segmentWidth) {
      //this loops over width
```

```
      //This will create a segment for each x and y position

      let segment = new ImageSegment(segXPos,segYPos,segmentWidth,segmentHeight);
      segments.push(segment);
    }
  }
}

function draw() {
  background(220);
  //lets draw the image to the canvas
  image(img, 0, 0);
  //lets draw the segments to the canvas,
  //we will use a for of loop to loop over the segments array
  for (const segment of segments) {
    segment.draw();
  }
}

//Here is our class for the image segments, we start with the class keyword
class ImageSegment {
  /*
  then we give the class a constructor, this is a special function that runs
  when we create a new instance of the class
  this constructor takes 4 parameters, the x and y position of the segment in the image
  and the width and height of the segment
  */
  constructor(srcImgSegXPosInPrm,srcImgSegYPosInPrm,srcImgSegWidthInPrm,srcImgSegHeightInPrm)
{
    //these parameters are used to set the internal properties of an instance of the segment
    //These parameters are named as imageSource as they are derived from the image we are usin
g
    this.srcImgSegXPos = srcImgSegXPosInPrm;
    this.srcImgSegYPos = srcImgSegYPosInPrm;
    this.srcImgSegWidth = srcImgSegWidthInPrm;
    this.srcImgSegHeight = srcImgSegHeightInPrm;
  }

  draw() {
    //Let's draw the segment to the canvas, for now we will draw it
    //as an empty rectangle so we can see it
    noFill();
    stroke(0);
    rect(this.srcImgSegXPos,this.srcImgSegYPos,this.srcImgSegWidth,this.srcImgSegHeight);
  }
}
```

Part 2:

Objective:

In this part of the tutorial we will sidestep for a second. We will leave the code we just wrote as is, and add a feature. We want to make a sketch that draws a circle wherever the mouse pointer is. The colour of the circle will come from the colour of the pixel in the image we loaded that is underneath the mouse pointer.

We are going to need to know how to get the colour of a part of an image, we can do this by using the image.get(x,y) method of an image, it will return a colour for any x,y coordinate of an image.

One important thing to note is that p5.js has a colour class built in so we don't have to manage any individual RGB values, the image.get(x,y) function will return a formatted colour object we can

use directly as a colour.

Challenges:

- Draw a circle that follows the mouse pointer.
- Get the colour of a pixel of an image from the location of the mouse pointer.

Solutions:

- We will use a p5.js colour object to store the colour in, this way we can set it when we need to and pass it to a fill() function to set the colour of a circle.
- The image class in p5.js has a built in function to get the colour given a set of pixel coordinates: get(xCoord, yCoord); this returns a p5.js colour object that we can use to set the colour.
- The built in mouseMoved() function in p5.js will run when the mouse is move. This lets us efficiently run code only when the mouse is moved, we will use this to update the colour only when the mouse moves.
- We will set the colour of the circle from our variable updated from the get() function inside our mouseMoved function, and use the built in mouseX and mouseY parameters to set the draw location of the circle.

Implementation:

```
//We need a variable to hold our image
let img;

//We will divide the image into segments
let numSegments = 50;

//We will store the segments in an array
let segments = [];

//Lets make a variable to store the colour of the pixel at the mouse position
//We will set it to black to start with using the built in color constructor
let pixelColour = null;

//lets load the image from disk
function preload() {
  img = loadImage('assets/Mona_Lisa_by_Leonardo_da_Vinci_500_x_700.jpg');
}

function setup() {
  //We will make the canvas the same size as the image using its properties
  createCanvas(img.width, img.height);
  //We can use the width and height of the image to calculate the size of each segment
  let segmentWidth = img.width / numSegments;
  let segmentHeight = img.height / numSegments;

  pixelColour = color(0);

  /*
  Divide the original image into segments, we are going to use nested loops
  lets have a look at how nested loops work
  first we use a loop to move down the image,
  we will use the height of the image as the limit of the loop
  then we use a loop to move across the image,
```

```
  we will use the width of the image as the limit of the loop
  Lets look carefully at what happens, the outer loop runs once, so we start at the top of the
image
  Then the inner loop runs to completetion, moving from left to right across the image
  Then the outer loop runs again, moving down 1 row image, and the inner loop runs again,
  moving all the way from left to right
  */

  for (let segYPos=0; segYPos<img.height; segYPos+=segmentHeight) {
    //this is looping over the height
    for (let segXPos=0; segXPos<img.width; segXPos+=segmentWidth) {
      //this loops over width
      //This will create a segment for each x and y position

      let segment = new ImageSegment(segXPos,segYPos,segmentWidth,segmentHeight);
      segments.push(segment);
    }
  }
}

function draw() {
  background(220);
  //lets draw the image to the canvas
  image(img, 0, 0);

  //lets draw the segments to the canvas,
  //we will use a for of loop to loop over the segments array
  for (const segment of segments) {
    segment.draw();
  }

  //Lets draw a circle with the colour of the pixel at the mouse position
  fill(pixelColour);
  stroke(255);
  circle(mouseX, mouseY, 40);

}

//We add the mouse moved function to get the colour of the pixel at the mouse position
function mouseMoved(){
  //lets get the colour of the pixel from the image at the mouse position
  pixelColour = img.get(mouseX, mouseY);
}

//Here is our class for the image segments, we start with the class keyword
class ImageSegment {
  /*
  then we give the class a constructor, this is a special function that runs
  when we create a new instance of the class
  this constructor takes 4 parameters, the x and y position of the segment in the image
  and the width and height of the segment
  */
  constructor(srcImgSegXPosInPrm,srcImgSegYPosInPrm,srcImgSegWidthInPrm,srcImgSegHeightInPrm)
{
    //these parameters are used to set the internal properties of an instance of the segment
    //These parameters are named as imageSource as they are derived from the image we are usin
g
    this.srcImgSegXPos = srcImgSegXPosInPrm;
    this.srcImgSegYPos = srcImgSegYPosInPrm;
    this.srcImgSegWidth = srcImgSegWidthInPrm;
    this.srcImgSegHeight = srcImgSegHeightInPrm;
  }

  draw() {
    //lets draw the segment to the canvas, for now we will draw it
    //as an empty rectangle so we can see it
    noFill();
    stroke(0);
    rect(this.srcImgSegXPos,this.srcImgSegYPos,this.srcImgSegWidth,this.srcImgSegHeight);
  }
}
```

Part 3:

Objective:

Now we know how to get the colour of a pixel, we will set the stroke colour of our segments to be the colour of the centre pixel of each segment. It will look similar to before, except with the segment borders different colours. This will also show us the differences that happen when we just take the colour from a centre point.

Challenges:

- We will need to give each segment a colour.
- We will need to use the colour in our class draw function to fill the rectangle.
- We will need to set the colour from the corresponding position in the image.

Solutions:

- We will add a colour parameter to our segment class as well as adding it in the constructor function, and pass the value from the constructor to the parameter.
- We will need to use the colour in our class draw function to set the stroke colour of the rectangle.
- We will set the colour when we initialise each segment in the setup, to do this we will use the colour at the centre of the segment. Using our nested for loops we already have the coordinates of the top right of each segment. We will get the colour from the centre by adding half the segment width to the x position and half the segment height to the y position.

Implementation:

```
//We need a variable to hold our image
let img;

//We will divide the image into segments
let numSegments = 50;

//We will store the segments in an array
let segments = [];

//lets load the image from disk
function preload() {
  img = loadImage('assets/Mona_Lisa_by_Leonardo_da_Vinci_500_x_700.jpg');
}

function setup() {
  //We will make the canvas the same size as the image using its properties
  createCanvas(img.width, img.height);
  //We can use the width and height of the image to calculate the size of each segment
  let segmentWidth = img.width / numSegments;
  let segmentHeight = img.height / numSegments;
  /*
  Divide the original image into segments, we are going to use nested loops
  */
```

```
  for (let segYPos=0; segYPos<img.height; segYPos+=segmentHeight) {
    //this is looping over the height
    for (let segXPos=0; segXPos<img.width; segXPos+=segmentWidth) {
      //We will use the x and y position to get the colour of the pixel from the image
      //lets take it from the centre of the segment
      let segmentColour = img.get(segXPos + segmentWidth / 2, segYPos + segmentHeight / 2);
       let segment = new ImageSegment(segXPos,segYPos,segmentWidth,segmentHeight,segmentColou
r);
       segments.push(segment);
    }
  }
}

function draw() {
  background(0);
  //lets draw the image to the canvas
  image(img, 0, 0);
  //lets draw the segments to the canvas
  for (const segment of segments) {
    segment.draw();
  }
}

//Here is our class for the image segments, we start with the class keyword
class ImageSegment {

  constructor(srcImgSegXPosInPrm,srcImgSegYPosInPrm,srcImgSegWidthInPrm,srcImgSegHeightInPrm,s
rcImgSegColourInPrm) {
    //these parameters are used to set the internal properties of an instance of the segment
    //These parameters are named as imageSource as they are derived from the image we are usin
g
    this.srcImgSegXPos = srcImgSegXPosInPrm;
    this.srcImgSegYPos = srcImgSegYPosInPrm;
    this.srcImgSegWidth = srcImgSegWidthInPrm;
    this.srcImgSegHeight = srcImgSegHeightInPrm;
    this.srcImgSegColour = srcImgSegColourInPrm;
  }

  draw() {
    //lets draw the segment to the canvas, for now we will draw it as an empty rectangle so we
can see behind it.
    //but this time the border will be the same colour as the centre.
    noFill();
    stroke(this.srcImgSegColour);
    rect(this.srcImgSegXPos, this.srcImgSegYPos, this.srcImgSegWidth, this.srcImgSegHeight);
  }
}
```

Part 4:

Objective:

In this part of the tutorial we want to end up with each image segment being drawn as a solid colour that we extract from the image and with a black outline so we can see the squares clearly. We also want to be able to toggle between the raw image and our segments.

Challenges:

- Make a boolean that can control what we see.
- Set the whole colour of the segment.
- Add an outline to the segments.

Solutions:

- We will create a drawSegments variable (a boolean) and create an if else statement in our draw loop controlled by drawSegments. If drawSegments is true we will iterate through the segments and draw them. If it is false we will just draw the raw image.
- We will add a stroke(0) function to our draw loop to add the border around the segments.
- We will use each segments colour variable to set the fill of each segments rectangle.

Implementation:

```
//We need a variable to hold our image
let img;

//We will divide the image into segments
let numSegments = 50;

//We will store the segments in an array
let segments = [];

//lets add a variable to switch between drawing the image and the segments
let drawSegments = true;

//lets load the image from disk
function preload() {
  img = loadImage('assets/Mona_Lisa_by_Leonardo_da_Vinci_500_x_700.jpg');
}

function setup() {
  //We will make the canvas the same size as the image using its properties
  createCanvas(img.width, img.height);
  //We can use the width and height of the image to calculate the size of each segment
  let segmentWidth = img.width / numSegments;
  let segmentHeight = img.height / numSegments;
  /*
  Divide the original image into segments, we are going to use nested loops
  */

  for (let segYPos=0; segYPos<img.height; segYPos+=segmentHeight) {
    //this is looping over the height
    for (let segXPos=0; segXPos<img.width; segXPos+=segmentWidth) {
      //We will use the x and y position to get the colour of the pixel from the image
      //lets take it from the centre of the segment
      let segmentColour = img.get(segXPos + segmentWidth / 2, segYPos + segmentHeight / 2);
       let segment = new ImageSegment(segXPos,segYPos,segmentWidth,segmentHeight,segmentColou
r);
       segments.push(segment);
    }
  }
}

function draw() {
  background(0);
  if (drawSegments) {
    //lets draw the segments to the canvas
    for (const segment of segments) {
      segment.draw();
    }
  } else {
    //lets draw the image to the canvas
    image(img, 0, 0);
  }
}
function keyPressed() {
  if (key == " ") {
    //this is a neat trick to invert a boolean variable,
```

```
    //it will always make it the opposite of what it was
    drawSegments = !drawSegments;
  }
}


//Here is our class for the image segments, we start with the class keyword
class ImageSegment {

  constructor(srcImgSegXPosInPrm,srcImgSegYPosInPrm,srcImgSegWidthInPrm,srcImgSegHeightInPrm,s
rcImgSegColourInPrm) {
    //these parameters are used to set the internal properties of an instance of the segment
    //These parameters are named as imageSource as they are derived from the image we are usin
g
    this.srcImgSegXPos = srcImgSegXPosInPrm;
    this.srcImgSegYPos = srcImgSegYPosInPrm;
    this.srcImgSegWidth = srcImgSegWidthInPrm;
    this.srcImgSegHeight = srcImgSegHeightInPrm;
    this.srcImgSegColour = srcImgSegColourInPrm;
  }

  draw() {
    //lets draw the segment to the canvas, now filled with the colour and with a black border
    stroke(0);
    fill(this.srcImgSegColour);
    rect(this.srcImgSegXPos, this.srcImgSegYPos, this.srcImgSegWidth, this.srcImgSegHeight);
  }
}
```

Part 5:

Objective:

In this last part of the tutorial we will add some interactivity. We will make the segments be scaled by how far away they are to the mouse cursor - the further away the bigger they are.

Challenges:

- Add some scaling to our class and use it in the draw function
- Calculate the distance from each segment to the mouse

Solutions:

- We will add a class member variable called scale, it will be initialised as 1, but we can set it externally.
- We will use the built in p5.js distance function - this function calculates the distance in pixels between two points.

Implementation:

```
//We need a variable to hold our image
let img;

//We will divide the image into segments
let numSegments = 50;

//We will store the segments in an array
let segments = [];
```

```
//lets add a variable to switch between drawing the image and the segments
let drawSegments = true;

//lets load the image from disk
function preload() {
  img = loadImage('assets/Mona_Lisa_by_Leonardo_da_Vinci_500_x_700.jpg');
}

function setup() {
  //We will make the canvas the same size as the image using its properties
  createCanvas(img.width, img.height);
  //We can use the width and height of the image to calculate the size of each segment
  let segmentWidth = img.width / numSegments;
  let segmentHeight = img.height / numSegments;
  /*
  Divide the original image into segments, we are going to use nested loops
  */

  for (let segYPos=0; segYPos<img.height; segYPos+=segmentHeight) {
    //this is looping over the height
    for (let segXPos=0; segXPos<img.width; segXPos+=segmentWidth) {
      //We will use the x and y position to get the colour of the pixel from the image
      //lets take it from the centre of the segment
      let segmentColour = img.get(segXPos + segmentWidth / 2, segYPos + segmentHeight / 2);
       let segment = new ImageSegment(segXPos,segYPos,segmentWidth,segmentHeight,segmentColou
r);
       segments.push(segment);
    }
  }
}

function draw() {
  background(0);
  if (drawSegments) {
    //lets draw the segments to the canvas
    for (const segment of segments) {
      //lets set the scale of each segment based on its distance from the mouse
       segment.scale = dist(segment.srcImgSegXPos, segment.srcImgSegYPos, mouseX, mouseY)/100;
       segment.draw();
    }
  } else {
    //lets draw the image to the canvas
    image(img, 0, 0);
  }
}
function keyPressed() {
  if (key == " ") {
    //this is a neat trick to invert a boolean variable,
    //it will always make it the opposite of what it was
    drawSegments = !drawSegments;
  }
}


//Here is our class for the image segments, we start with the class keyword
class ImageSegment {

  constructor(srcImgSegXPosInPrm,srcImgSegYPosInPrm,srcImgSegWidthInPrm,srcImgSegHeightInPrm,s
rcImgSegColourInPrm) {
    //these parameters are used to set the internal properties of an instance of the segment
    //These parameters are named as imageSource as they are derived from the image we are usin
g
    this.srcImgSegXPos = srcImgSegXPosInPrm;
    this.srcImgSegYPos = srcImgSegYPosInPrm;
    this.srcImgSegWidth = srcImgSegWidthInPrm;
    this.srcImgSegHeight = srcImgSegHeightInPrm;
    this.srcImgSegColour = srcImgSegColourInPrm;
    this.scale = 1;
}

  draw() {
    //lets draw the segment to the canvas, now filled with the colour and with a black border
```

```
    stroke(0);
    fill(this.srcImgSegColour);
    rect(this.srcImgSegXPos, this.srcImgSegYPos, this.srcImgSegWidth * this.scale, this.srcImgSegHeight * this.scale);
  }
}
```

Extra challenge:

This can help you with the quiz this week - there is something wrong with the segments, do you know what it is?

Inspiration:

This weeks inspiration is the work of artist Petros Vrellis. Petros has made some amazing work manipulating existing images. Here are two examples:

'Out of all things one, and out of one all things' **http://artof01.com/vrellis/works/AllAndOne.html (http://artof01.com/vrellis/works/AllAndOne.html)**

This phenomenal work uses colour blending and separation to break images into layers. Each layer has its own unique image, but as they are added together, the layer images contribute to a new and evolving image. This work is presented as analogue, with the layers printed onto transparent sheets, however the layers were made by a complex computational process.

The second work is called 'Starry night' **http://artof01.com/vrellis/works/starry_night.html (http://artof01.com/vrellis/works/starry_night.html)**

(http://artof01.com/vrellis/works/starry_night.html) This work is actually an iPad app that brings Vincent van Gogh's starry night to life with clever animations. This work is complex, and relies on mathematical fluid simulations responding to touch input, but produces beautiful results.

Homework:

As well as the quiz this week you have some other homework. You will need to make a GitHub account for next weeks tutorial. Go to **https://github.com (https://github.com)** and use your USYD email to make an account.

Make sure you keep the password somewhere easy to find and check your account works by logging out and logging in again.

Happy coding